

## Lecture -3 Java Variables & Datatypes

**Variable** → A variable is a name given to a memory address where a value is stored. Always give meaningful names to variable.

$a = 5;$   
variable

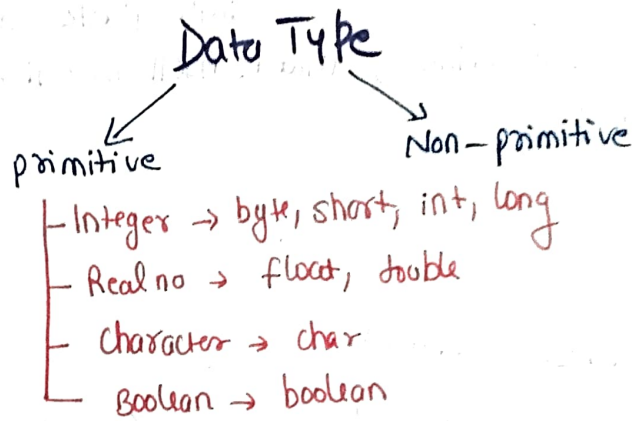
\* **Identifier** → Its name given to any class, function, variable etc.

Syntax → datatype identifier = value;

→ Datatype specifies the type of data a variable will be holding or a method will be returning.

→ Java is statically typed as we need to specify datatype of a variable.

→ we use ; to specify line end.



### \* Integers

1.) byte → -128 to 127

2.) Some other types have huge ranges as

hierarchy increases

from short → long

→ Every no is signed in java that is we can represent both -ve & +ve no so if in byte it is of 8 bits and if it is signed so

MSB is bit to represent whether it is -ve or +ve & other 7 bits represent value

short →

long → 8 byte  
int → 4 byte  
short → 2 byte  
byte → 1 byte

$(-2^7) - (-2^7 - 1) \rightarrow$  general formula  $(-2^n) - (-2^n - 1)$

→ Decimal no system → base 10  
(10 unique characters/ numbers to represent value)

→ Some Binary no system with base 2

## \* Real Numbers (floating point numbers)

↳ float (64 bits)  $4.9e-324$  to  $1.8e+308$  (8 bytes)  
↳ double (32 bits)  $1.4e-045$  to  $3.4e+038$  (4 bytes)

- float (single precision) → previously hardware was optimized to process this in an optimized & fast way.

- double (double precision) → now modern hardware are more optimized to process this double faster than float.

## \* Characters

↳ char (this can represent only unicode character).

char alpha = 'a'; → it will have only one character with single quotes only.

we map these characters with an integer that is ASCII or unicode table & compiler stores that no in memory.

→ char takes 2 bytes

## \* Boolean

↳ true / false (1 byte)

→ In Java we cannot do this boolean a = 6; X error as we only have to (true / false) values.

\* Literal → value assigned to a variable.

→ In Java to represent float value we need to use character 'f'

float b = 42.69f;

because since for modern hardware double is more optimized, Java wants you to use double. Without this 'f' it will throw error as it means we are storing double value to float.

\* Representing no in binary, octal, hexo decimal.

1) Binary → a = 0b101

→ sout(a) → 5

2) Octal → a = 07

→ sout(a) → 7

3) Hexadecimal → a = 0xA

→ sout(a) → 10

a = 0x9

→ sout(a) → 9

but while printing it will print in integer only

eg → int x = 0b101;  
sout(x);  
→ 5

these notations can be used for real numbers as well like float / double.



## Notation to represent scientific way of representing numbers

int a = 4;      float f = 4f;  
int c =  $4e^{10} \rightarrow 4 \times 10^{10}$       float c =  $6.2e^{23} = 6.2 \times 10^{23}$ .

→ to make int, long, float, double more readable we can add / use ( ) b/w digits to make it more readable but just before / after decimal (.) or (e)  
~~XXXXXX XXXXXX XXXXXX XXXXXX XXXXXX XXXXXX XXXXXX XXXXXX XXXXXX XXXXXX~~ we cannot do it.

eg → long l = 426\_45924\_106 ✓

float f = 42\_6.92\_68 ✓

float k = 49\_26.46e9\_26 ✓

double d = 4\_92e\_62 ✗

→ In Java all whole numbers are treated as int by default, even if you store them in long. So initially it treats it as int, then check can it fit in int? If in range then no error is thrown, else error comes. To tell its long use long literal (L) ⇒ long d = 234424\_2424232L;

→ Underscore gets ignored by compiler. Its only for readability.

## \* Variable Declaration + Defining

int x; → variable declaration (it has some garbage value).

int x = 10; → variable definition / variable initialization

x = 20; → variable assignment

we cannot do this  
int x; line 7  
Sout(x) → error lines  
because Java do not want to read some garbage value

\* Keywords → keywords are reserved words in Java that have a predefined meaning to compiler. They cannot be used as variable names, class names or method names.

eg → void, static, public, class etc.

→ we have a total of 68 keywords in Java. Java reserved 2 keywords that were never used.

→ goto, const

## \* Comments In Java

int x;  
x = 10;  
Sout(x) → this is valid because now you wouldn't read garbage value